



Projeto Guiado: Biblioteca

Aula 12 · Bloco 3 — POO na Prática

Tudo do curso até aqui, num sistema só — em 6 etapas

Esta aula é diferente

Não há conceito novo.

Há **um sistema para construir**, uma etapa por vez, com um **commit ao fim de cada uma**.

É a ponte entre *"sei o conceito"* e *"consigo fazer"*.

O que vamos construir

Um sistema de biblioteca em console: cadastrar itens e usuários, emprestar, devolver, listar. As regras:

-  Cada usuário tem um **tipo** que define quantos itens pega e por quantos dias;
-  Item emprestado **não** pode ser emprestado de novo;
-  Devolução com atraso gera **multa de R\$ 1,00 por dia**;
-  **Nenhuma entrada do usuário** pode derrubar o programa.

A estrutura de destino

```
biblioteca/  
├── model/      Emprestavel (interface) · ItemAcervo (abstrata)  
│              Livro · Revista · TipoUsuario (enum)  
│              Usuario · Emprestimo  
├── service/    BibliotecaService + excecoes/ (4 próprias)  
└── app/        Main.java
```

O diagrama de classes está na aula 11. Abra numa aba ao lado.



Como usar esta aula

Faça uma etapa.

Rode. Veja funcionando. Commite.

Só então avance.

Se pular etapas, a próxima não fecha.

Etapa 1 — O modelo básico

```
public abstract class ItemAcervo {  
    protected String codigo;  
    protected boolean disponivel = true;  
  
    /** Cada tipo de item se descreve à sua maneira. */  
    public abstract String descricao();  
}
```

Comece pelo **domínio**, sem menu e sem serviço. `ItemAcervo` é abstrata: "*item genérico*" não existe na prateleira. Depois `Livro` e `Revista`.

Etapa 2 — Encapsulamento e regras no lugar certo

Atributos `private` ou `protected`. Getters só do que faz sentido expor.

E as regras do objeto **dentro do objeto**:

- `Usuario` sabe dizer se **pode pegar mais** itens;
- `TipoUsuario` (o `enum`) carrega o limite e os dias;
- `ItemAcervo` sabe se **está disponível**.

💡 Se você precisou de um `if` fora da classe para responder algo sobre ela, provavelmente falta um método lá dentro.

Etapa 3 — A interface **Emprestavel**

```
public interface Emprestavel {  
    void emprestar();  
    void devolver();  
    boolean estaDisponivel();  
}
```

Aqui entra a **ObraDeReferencia**: um item do acervo que **não pode ser emprestado**.

💡 É o momento em que o desenho prova seu valor: o tipo novo entra sem quebrar nada do que já existe.

Etapa 4 — O **BibliotecaService**

A camada que **orquestra**. Guarda as coleções e oferece as operações:

```
public class BibliotecaService {  
    private List<ItemAcervo> acervo = new ArrayList<>();  
    private List<Usuario> usuarios = new ArrayList<>();  
  
    public void cadastrar(ItemAcervo item) { ... }  
    public ItemAcervo buscarPorCodigo(String codigo) { ... }  
}
```

⚠ Nenhum `System.out.println` aqui dentro. A regra de ouro da aula 11 vale a partir de agora.

Etapa 5 — Empréstimo, devolução e exceções

O coração do sistema. Quatro exceções próprias:

```
ItemNaoEncontradoException  
ItemIndisponivelException
```

```
UsuarioNaoEncontradoException  
LimiteExcedidoException
```

O **service** **lança**. O **app** **traduz** para o usuário.

💡 É a divisão de responsabilidades da aula 10 num sistema de verdade — e o motivo pelo qual o mesmo **service** serviria a um app de celular.

Etapa 6 — O menu

`do-while` com `switch`, na camada `app`. Aqui, e **só aqui**, moram o `Scanner` e o `System.out`.

```
do {
    exibirMenu();
    try {
        opcao = Integer.parseInt(scanner.nextLine());
        switch (opcao) { ... }
    } catch (Exception e) {
        System.out.println("⚠ " + e.getMessage());
    }
} while (opcao != 0);
```

Nenhuma entrada pode quebrar o programa. Teste digitando letra onde se espera número.



Para fechar

- ☐ As 6 etapas concluídas, **cada uma com seu commit**;
- ☐ Nenhum `System.out` fora de `app` ;
- ☐ O menu sobrevive a qualquer entrada — teste com letra, vazio e número negativo;
- ☐ README do projeto explicando como compilar e rodar.



Fim do Bloco 3

Você tem um sistema com camadas,
exceções próprias e polimorfismo de verdade.

Bloco 4 — Java moderno

Arquivos, lambdas e Streams.